Remote Client Telemetry for Repository-Level Coding Agents: A Closed-Loop Debugging Protocol

Abstract

Repository-level coding agents can read source code and run local tests, but they cannot observe failures on a user's remote device. We present **Remote Telemetry Tool**, a prototype architecture that gives a coding agent task-scoped client telemetry and closes the loop between diagnosis, repair, and device-side verification. It combines bounded, redacted event retrieval; a server-enforced case state machine that the agent cannot skip and cannot approve on the user's behalf; and runtime-selectable instrumentation probes published as data and withdrawn at cleanup, so a client emits nothing until an agent asks for evidence. The artifact contains a Python tool library, a Supabase schema with security-definer RPCs, a Google OAuth backend, a dashboard, an agent runner, and a read-only evaluation harness. Verification is limited to 54 local unit tests (all passing) over mocked tool behavior, redaction, and state-machine rules, which establish neither deployment security nor end-to-end reliability. An earlier three-task run is invalid for efficacy claims because it reused live case identifiers, allowed state-mutating calls, and used unadjudicated heuristics, so we report no benefit. We contribute the protocol, its safety boundary, and a preregistered paired evaluation.

1. Introduction

Coding agents commonly operate inside a repository sandbox, which supports file inspection, code edits, and local test execution but does not expose runtime behavior from a user's phone or another remote client. A report such as "the highlight palette does not open" is under-specified: the failure could arise from a missing state transition, a presentation-context issue, an authorization response, or a device-specific condition.

Remote telemetry could narrow that search, but giving an agent broad database access is unsafe and can flood a limited context window with noisy or sensitive events. The central question is therefore:

> ***Can a constrained, task-scoped telemetry tool give repository-level coding agents actionable remote-client evidence without widening data exposure, and does that evidence improve debugging outcomes?***

Our contribution is a prototype architecture and an evaluation proposal, not a demonstrated result.

1. **A closed debugging loop.** A case object carries evidence from a user's report to an agent's proposal, and a server-enforced state machine separates *evidence collected* from *repair validated*. The agent cannot skip the user, and it cannot approve a fix on the user's behalf.
2. **Reversible, runtime-selectable instrumentation.** Probes are declared as data and published to a case, so an agent can refine what it observes without shipping a new build per iteration, and instrumentation can be withdrawn when the case closes. The default client emits nothing.
3. **An explicit verification boundary.** We separate what the artifact implements from what it has not been shown to do, and register a paired evaluation with blinded adjudication before any efficacy claim is made.

The artifact is implemented in Python, SQL, JavaScript, and JSON; all 54 local tests pass, but they are mocked unit tests over the rules, not deployment security or end-to-end reliability.

2. Related Work

The references below are representative rather than a systematic survey.

Repository-level agents and benchmarks. SWE-bench [1] evaluates whether models can resolve real GitHub issues from an issue description and a repository snapshot. SWE-agent [2] shows that the agent-computer interface, not only the model, materially changes agent performance, and Agentless [3] shows that a simple localization-repair-validation pipeline can beat complex agent scaffolds. All three observe failures through the repository and the test harness; a user-visible defect on a physical device is outside that surface.

Interleaved reasoning, acting, and open-weight models. ReAct [4] established the pattern of interleaving reasoning traces with tool actions, the pattern our protocol instantiates for telemetry access, and Google ADK [5]

provides the orchestration and function-calling machinery such a protocol would be hosted in. Gemma 4 [6] spans 2.3B to 31B parameters with native function calling and on-device-oriented small variants, so context economy is a first-order concern, motivating bounded, case-scoped retrieval instead of broad log search.

Data plane and gap. Row Level Security and Postgres function privileges are the standard way to restrict who may read which rows [7, 8]; we apply the same idea one level up, giving the agent a narrow tool surface while the database enforces ownership per case. Our contribution is not the event format, which is established practice, but the closed protocol around it and its reversibility: we are not aware of work that treats the end user's device as an explicit participant in an agentic debugging loop with a server-enforced protocol that separates evidence from repair validation and guarantees withdrawable instrumentation. That protocol, and a test of whether it helps, is what this artifact proposes.

3. System Design and Method

3.1 Client event capture

An instrumented client emits a compact event when a relevant operation fails: a bounded event name, a bounded error category, app and build version, OS version, and a correlation identifier. It excludes raw user content, access tokens, and unrestricted device data. By default the client collects nothing; capture is attached to an active probe for a specific case.

3.2 Storage and access control

The Supabase schema defines app-owned `public.users`, `public.app_sessions`, `public.client_logs`, `public.debug_cases`, `public.case_probes`, `public.case_reports`, `public.tool_api_keys`, and `public.ai_sessions`. A `client_logs` row carries a bounded `level`, a bounded `event` name, a JSONB `detail` object, optional user-agent metadata, a server-generated timestamp, and a nullable `case_id` linking the event to the debugging attempt that requested it; a composite `(user_id, created_at DESC)` index supports recent-event queries, and `case_id` filtering keeps case reads cheap.

The schema is not by itself a security boundary. Every access goes through security-definer RPCs that hash the presented key, resolve it to an owning `user_id`, and verify that the requested case belongs to that user before returning or mutating anything; the functions pin `search_path = ''` and have execute privileges revoked from `public`, `anon`, and `authenticated` before targeted grants. The backend holds a server-only Supabase secret key that bypasses RLS, so the rules currently live in application code and RPC bodies; deployment must add independent defense in depth (RLS policies, restricted schema, egress rules) and must never expose that key to a client or to model context.

3.3 Agent tool boundary

The agent never receives arbitrary SQL or unrestricted credentials. `telemetry_tool.py` exposes one agent-facing function per capability; each validates its arguments locally, calls the management API with a developer key, and returns JSON to the model (Table 1).

Tool (agent-facing function)	Bounds enforced before the call
`list_debug_cases_for_agent(limit, state)`	`limit` 1..100; known `state`
`fetch_case_logs_for_agent(case_id, limit)`	UUID `case_id`; `limit` 1..100
`fetch_recent_client_logs_for_agent(limit, client_session_id)`	`limit` 1..100; optional session filter
`request_instrumentation_for_agent(case_id, instructions, probes)`	<=20 probes; `probe_id` <=80 chars; `target` <=200; `level` in {info, warn, error}; `capture` an object
`await_user_recheck_for_agent(case_id, instructions, probes)`	Same probe bounds; probes optional
`finalize_case_for_agent(case_id)`	Only from `rechecked`
`abandon_case_for_agent(case_id)`	Only from non-terminal states
`cleanup_case_for_agent(case_id)`	Only from `approved`; deletes telemetry and deactivates probes

Table 1. Agent-facing tool surface and the bounds enforced before each call.

Output is bounded as well as input. Detail objects are redacted recursively against a fixed set of sensitive keys (email, phone, token, password, authorization, access and refresh tokens, secret, ssn, credit card); free-text fields (title, user report, instructions, report, name) are scrubbed for e-mail addresses and long digit runs and truncated at 4000 characters; other strings are truncated; row count and total response size are capped (256 KB). These are defense-in-depth measures, not a privacy guarantee.

Tool output is untrusted diagnostic data: log strings can contain secrets or prompt-injection text, so the agent must treat retrieved events as evidence, never instructions. Every call is authenticated by key and scoped to one account, and the model cannot widen that scope because filters are validated and ownership re-checked server-side.

Figure 1 shows the surface the agent can reach and the boundary it cannot cross.

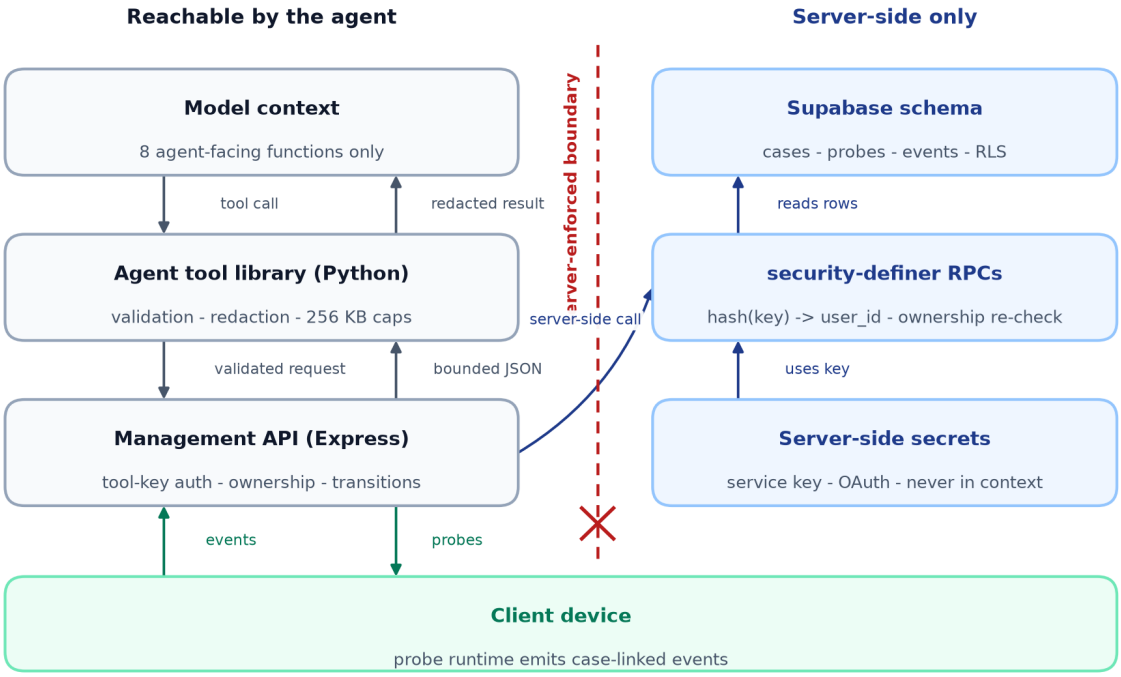


Figure 1. Trust boundaries: the agent reaches only the tool surface, while ownership checks, redaction, and the database stay server-side.

3.4 The closed debugging loop

Read access alone does not close the loop: an agent can observe an event but cannot ask the device for better evidence, cannot know whether the reproduction attempt succeeded, and cannot verify that a patch fixed the symptom. The protocol is therefore bidirectional, stateful, and human-in-the-loop.

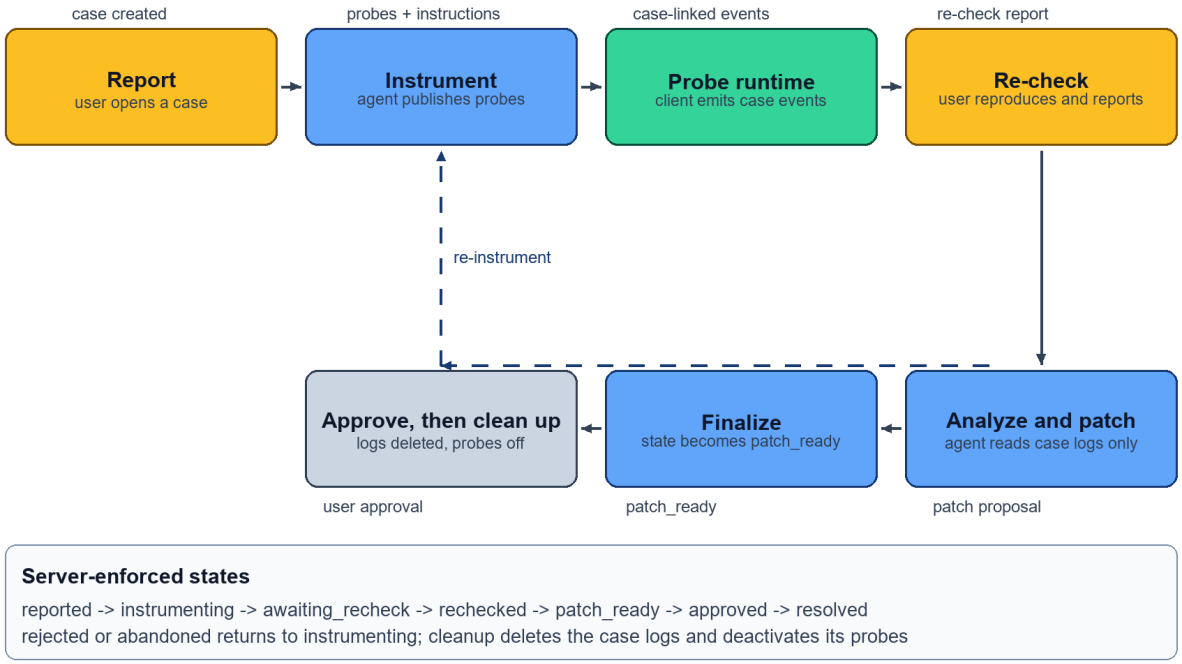


Figure 2. The closed debugging loop: evidence collection is separated from repair validation, and instrumentation is withdrawn at cleanup.

1. **Report.** The client creates a `debug_cases` row with the failure title and the user's own description. No telemetry is captured by default.
2. **Instrument.** The agent publishes probes (`probe_id`, `target`, bounded `capture` object, `level`) plus instructions; the client applies them at runtime and links every resulting event to the case, so evidence is scoped to one attempt.
3. **Await re-check.** The agent tells the user what to try and the case moves to `awaiting_recheck`; the client polls case state with a version counter, so no push infrastructure is required.
4. **Re-check.** The client reports its outcome through `report_case_progress`, which appends a bounded entry to `case_reports` and moves the case to `rechecked`; a dedicated timeline table keeps repeated re-checks from overflowing the original 4000-character report column.
5. **Analyze and patch.** The agent reads only case-scoped evidence, separates observation from inference, inspects the repository, and proposes a minimal patch. Without a faithful client test, the agent reports that limitation instead of claiming the remote issue is fixed.
6. **Finalize.** The agent calls `finalize_case`, moving the case to `patch_ready`; it cannot skip this state and cannot approve on the user's behalf.
7. **Approve and clean up.** The user approves or rejects through `approve_case`. Only after approval may the agent call `cleanup_case`, which deletes the case's telemetry, deactivates its probes so the device stops collecting, and marks the case `resolved`.

Table 2 gives the state machine, enforced twice in the Express layer and in the RPCs, so a disallowed transition yields an error rather than an inconsistent record.

State	Meaning	Allowed next states
`reported`	Failure reported	`instrumenting`, `abandoned`
`instrumenting`	Probes live	`awaiting_recheck`, `instrumenting`, `abandoned`
`awaiting_recheck`	Reproduction asked	`rechecked`, `instrumenting`, `abandoned`
`rechecked`	Outcome in	`patch_ready`, `instrumenting`, `abandoned`
`patch_ready`	Fix ready	`approved`, `rejected`, `instrumenting`, `abandoned`
`approved`	Fix accepted	`resolved`

State	Meaning	Allowed next states
`rejected`	Fix refused	`instrumenting`, `abandoned`
`resolved`	Cleaned up	none (terminal)
`abandoned`	Ended	none (terminal)

Table 2. Case state machine enforced in the Express layer and in the Supabase RPCs.

Figure 3 draws the same machine as a transition graph, including the returns to `instrumenting` and the terminal states.

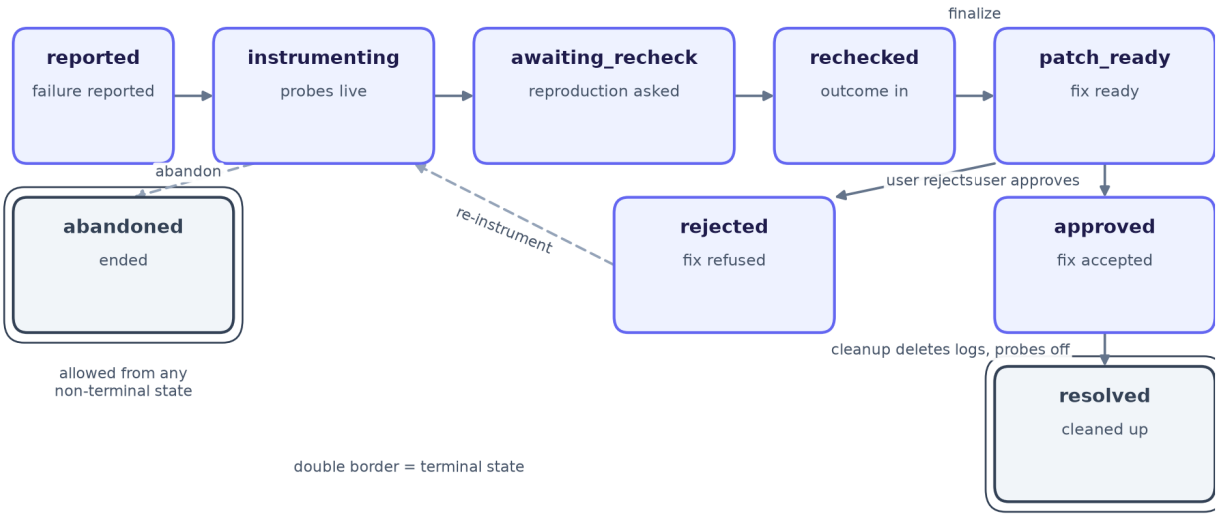


Figure 3. The server-enforced case state machine. Transitions are validated twice, in the Express layer and in the RPCs, and only the user can approve a fix.

Two properties carry the contribution. **Runtime-selectable probes** decouple instrumentation from the deployment cycle, so diagnostic iterations do not require a new build and the baseline client emits nothing until a probe is active. **Reversible instrumentation** follows from probes being data rather than inlined source: they can be withdrawn when the case closes, which is what makes the cleanup step meaningful rather than aspirational.

3.5 Illustrative scenario: iOS highlight palette

A user selects text and taps the highlight control, but the color palette never appears. The client records a structured event such as:

```
{"level": "error", "event": "highlight_palette_presentation_failed",
 "detail": {"view": "HighlightPaletteView", "failure_code": "missing_presentation_context",
            "app_version": "1.2.0", "os_version": "iOS 18"}}
```

The agent retrieves only events authorized for that case, searches the repository for the palette presentation path, examines state transitions and presentation context, and updates a regression test where feasible. The event guides the search but does not prove the root cause: instrumentation may be incomplete, the event may be stale, or the failure may have another cause. A valid repair claim requires a test or a controlled reproduction.

4. Implementation

The artifact has five implementation parts.

Agent tool library. `telemetry_tool.py` is a thin Python client that validates arguments (UUIDs, limits, probe shapes), posts to the management API, and returns JSON strings. It never receives a Supabase key: keys come from the environment, and session keys are hidden from model-visible output.

Database. `supabase_setup.sql` and `migration_debug_cases.sql` define the schema, RPCs, grants, and case tables; `lint_schema.py` checks foreign-key column ordering before a migration is applied, and `check_schema.py` probes a deployed schema for the expected tables, functions, and privileges.

Management backend. `web_demo/server/server.js` is an Express server that validates Google OAuth, maintains app-owned users and opaque application sessions, separates developer keys from client ingest keys (distinct prefixes, hashed at rest, individually revocable), and exposes the case lifecycle over HTTP: `/api/client/cases/*` for the client and `/api/tool/cases/*` for the agent. Unauthenticated dashboard access returns `401 Sign in required`; agent routes require a tool key and a same-origin check.

Dashboard and agent. `web_demo/src/main.js` is a Vite dashboard for Google sign-in, key and session management, case listing, probe inspection, re-check feedback, and approval; all data comes from the backend. `ollama_agent.py` exposes the protocol as OpenAI-compatible function tools for an Ollama-hosted agent, with workspace read, patch, and test tools, and `eval/eval_runner.py` is a read-only pilot harness.

The device side is specified in `IOS_HIGHLIGHT_SCENARIO.md` and exercised by `ai_client_demo/debug_loop_demo.py`, a scripted client that drives the loop against a running backend. No production iOS client with a probe runtime is part of this artifact, so the "client applies the probes" step is scripted, not shipped.

5. Experiments and Evaluation Protocol

5.1 Verification performed

Table 3 reports the local test suites, which cover argument validation and bounds, redaction and free-text scrubbing, probe normalization and rejection, the transition map, terminal states, session tokens, and dashboard decision logic. They are deterministic unit tests over mocked clients.

Suite	Command	Tests	Result
`web_demo/src/case_actions.test.js`	`npm run test:ui`	7	pass
`web_demo/server/server.test.js`	`npm run test:api`	21	pass
`telemetry_tool_poc/test_telemetry_tool.py`	`python -m unittest discover -s telemetry_tool_poc`	12	pass
`test_ollama_agent.py`	`python -m unittest test_ollama_agent`	14	pass

Table 3. Local test suites and their results.

These suites are not a security audit, not a proof of correctness under concurrency or network partitions, and not evidence that a deployed system behaves as the mocks do; the loop has been driven against a local backend, not a deployed Supabase project with production OAuth.

5.2 An exploratory run that we do not report as evidence

A three-task exploratory run with `gemma4:31b:cloud` is retained in `eval/results/eval_summary.json` for auditability only and is marked invalid for publication: the first runner counted any tool call as a resolution, inferred accuracy with a substring heuristic, reused live case identifiers, and allowed state-mutating tools, so it cannot establish diagnosis accuracy, repair, protocol compliance, or a paired comparison.

5.3 Preregistered paired study

A valid follow-up study should use isolated synthetic cases and the same model, repository snapshot, prompt, and compute budget in both conditions: **(A)** issue and repository access only; **(B)** identical access plus read-only, case-scoped telemetry. Run order is randomized and each task is repeated for model variability. Task descriptions, root causes, relevant evidence, and validation tests are frozen before the run, the agent is kept away from answer keys, and both conditions receive equivalent context and tool budgets. Repair success requires an independently executed task-specific test or reproduction; root-cause accuracy requires blinded adjudication against a prespecified rubric. Outcomes are analyzed pairwise with uncertainty intervals and exclusions.

The harness already supports part of this protocol: it permits case listing and case-log retrieval only, records model-reported tokens, latency, and tool errors, leaves `diagnostic_accuracy` and `resolution_success` unset for independent adjudication, refuses to touch telemetry without an explicit `--confirm-nonproduction` flag, and labels its output `pilot_unadjudicated`. The bundled scenario file contains three synthetic task descriptions and no live case identifiers, so future runs should use isolated fixtures. Planned outcomes are adjudicated accuracy,

validated repair, leakage, protocol compliance, tool-call count, latency, and token usage, plus a separate adversarial security evaluation covering cross-user and cross-case reads, malformed probe payloads, prompt injection inside event fields, unauthorized cleanup, and authorization failures at every key type. No performance gain is claimed until a controlled experiment is completed.

6. Limitations and Risks

Telemetry can be absent, delayed, duplicated, misleading, or too coarse to reproduce a UI defect. Identifiers, user descriptions, and stack traces may be sensitive, and redaction is a mitigation rather than a guarantee. Remote storage adds network dependency, latency, cost, retention obligations, and an attack surface, and log content is attacker-controllable, so it must never be treated as trusted instructions. A retrieval tool is not an autonomous repair system: reliable repair also needs repository access, tests, authorization controls, and validation. The device-side probe runtime is specified and scripted but not shipped, so the loop's final link is the least exercised part of the design.

7. Conclusion

Task-scoped remote telemetry closes the information gap between a local coding agent and a failure that only happens on the user's device. The design combines bounded, redacted retrieval with a server-enforced protocol that separates evidence collection from repair validation, and with instrumentation that can be added and withdrawn at runtime. The artifact implements that protocol across a tool library, schema, OAuth backend, dashboard, agent runner, and read-only evaluation harness; 54 local tests pass, and no deployment or efficacy claim is made. Next steps: deploy and review the OAuth and key model, validate the tool in the target agent harness, build reproducible client-debugging tasks, and run the paired evaluation.

References

1. C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR 2024. arXiv:2310.06770.
2. J. Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." NeurIPS 2024. arXiv:2405.15793.
3. C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based Software Engineering Agents." FSE 2025. arXiv:2407.01489.
4. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023. arXiv:2210.03629.
5. Google, "Agent Development Kit (ADK) documentation." https://google.github.io/adk-docs/
6. Gemma Team, "Gemma 4 Technical Report," 2026. arXiv:2607.02770.
7. Supabase, "Row Level Security." https://supabase.com/docs/guides/database/postgres/row-level-security
8. Supabase, "Database functions." https://supabase.com/docs/guides/database/functions

Reproducibility

The artifact ships the tool library, schema and migrations, linter and checker, Express backend, Vite dashboard, agent runner, pilot harness, task fixtures, and this write-up with its source. Test suites run with `npm test` in `web_demo/`, `python -m unittest discover -s telemetry_tool_poc`, and `python -m unittest test_ollama_agent`; the dashboard starts with `npm start` in `web_demo/` (default port 5175), and the closed loop can be driven with `python -m ai_client_demo.debug_loop_demo`. The pilot harness requires an explicit confirmation flag and isolated fixtures before it touches any endpoint.